

A Major Project Report
On
**Automated Road Damage Detection Using UAV
Images and Deep Learning Techniques**

*Submitted to CMR ENGINEERING COLLEGE
In Partial Fulfillment of the requirements for the Award of Degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

Submitted
By

S.Shashank (228R1A05J0)

Under the Esteemed guidance

Dr. Praveen Chouksey

Associate Professor, Department of CSE



Department of Computer Science & Engineering
CMR ENGINEERING COLLEGE
UGC AUTONOMOUS

**(Approved by AICTE, NEW DELHI, Affiliated to JNTU, Hyderabad)
Kandlakoya, Medchal Road, R.R. Dist. Hyderabad-501 401)
2025-2026**

CMR ENGINEERING COLLEGE

UGC AUTONOMOUS

*(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad)
Kandlakoya, Medchal Road, Hyderabad-501 401*

Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the project entitled “**Automated Road Damage Detection Using UAV Images and Deep Learning Techniques**” is a bonafide work carried out by

S.Shashank **(228R1A05J0)**

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this Major project have been verified and are found to be satisfactory. The results embodied in this Major project have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide

Major Project Coordinator

Head of the Department

Dr.Praveen Chouksey

Dr.G. Sumalatha

Dr. Sheo Kumar

Associate Professor

Associate Professor

Professor & H.O.D

CSE Department

CSE Department

CSE Department

CMREC

CMREC

CMREC

External Examiner

DECLARATION

This is to certify that the work reported in the present Major project entitled "**Automated Road Damage Detection Using UAV Images and Deep Learning Techniques**" is a record of bonafide work done by us in the Department of Computer Science and Engineering, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Major project stage1 report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

S.Shashank (228R1A05J0)

ACKNOWLEDGMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Sheo Kumar**, HOD, **Department of CSE, CMR Engineering College** for their constant support.

We are extremely thankful to **Dr. Praveen Chouksey**, Associate Professor, Internal Guide, Department of CSE, for his/ her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We thank **Dr.G.Sumalatha**, Associate Professor, CSE Department, Major Project Coordinator for her constant support in carrying out the project activities and reviews.

We express my thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, We are very much thankful to our parents who guided me for every step.

S.Shashank

(228R1A05J0)

CONTENTS

TOPIC	PAGENO
List of Figures	i
List of Tables	ii
ABSTRACT	iii
1. INTRODUCTION	
1.1. Introduction & Objectives	1
1.2 .Purpose of the Project	3
1.3 Existing system & Disadvantages	3
1.4 Proposed system with Features	4
2. LITERATURE SURVEY	6
3. SOFTWARE REQUIREMENT ANALYSIS	
3.1. Problem Specification	9
3.2. Modules and their Functionalities	10
3.3. Functional Requirements	11
3.4 .Non Functional Requirements	11
3.5. Feasibility Study	11
4. SOFTWARE & HARDWARE REQUIREMENTS	
4.1. Software requirements	13
4.2. Hardware requirements	13
5. SOFTWARE DESIGN	
5.1 System Architecture	14
5.2 Data flow Diagram	16
5.2.1 Use case Diagram	17
5.2.2 Activity Diagram	18
5.2.3 Sequence Diagram	19
5.2.4 Class Diagram	20
5.3 ER Diagram	21
6. CODING AND IMPLEMENTATION	
6.1 Sample Code	23
6.2 Connectivity	48
6.3 Methodologies	49

7. SYSTEM TESTING	
7.1 Types of Testing	51
7.2 Test Objectives	52
7.3 Test Strategies	53
7.4 Test Cases	54
7.5 Test Results	55
8. OUTPUT SCREENS	
8.1 Image Upload Interface	58
8.2 Prediction Result – Good Road Condition	59
8.3 Prediction Result – Satisfactory Road Condition	60
8.4 Prediction Result – Very Poor Road Condition	61
9. CONCLUSION	62
10. FUTURE ENHANCEMENT	63
11. REFERENCES	64

LIST OF FIGURES

S.NO	DESCRIPTION	PAGENO
1.4.1	Comparison Diagram	5
5.1	Architecture Diagram	15
5.2	Data Flow Diagram	16
5.2.1	Use case Diagram	17
5.2.2	Activity Diagram	18
5.2.3	Sequence Diagram	19
5.2.4	Class Diagram	20
5.3	ER Diagram	21
8.1	Image Upload Interface	58
8.2	Prediction Result – Good Road Condition	59
8.3	Prediction Result – Satisfactory Road Condition	60
8.4	Prediction Result – Very Poor Road Condition	61

LIST OF TABLES

S.NO	DESCRIPTION	PAGENO
1	Literature Survey	8
2	Test cases Table	56

ABSTRACT

Maintaining road infrastructure is essential for ensuring safe and efficient transportation. Traditional manual road inspections are labor-intensive, time-consuming, and prone to human error. To overcome these challenges, deep learning and Unmanned Aerial Vehicle (UAV) technologies have emerged as effective solutions for automated road damage detection. The existing system employs YOLO-based architectures (YOLOv4, YOLOv5, and YOLOv7) to detect and classify various pavement defects such as longitudinal cracks, transverse cracks, alligator cracks, potholes, and repairs from UAV imagery. While these models achieved notable detection accuracy—up to 73.2% mAP@0.5 using YOLOv7—the reliance on bounding-box detection limits precision for fine cracks and increases computational demand. To address these issues, a ResNet101-based proposed system is introduced. The proposed method utilizes a two-stage pipeline: patch-wise classification and regional refinement to produce detailed damage heatmaps instead of simple bounding boxes. This approach enhances sensitivity to minor cracks, improves robustness under varying lighting conditions, and provides more accurate spatial mapping of damaged areas. Experimental validation is expected to show improved classification accuracy and geospatial reporting efficiency, paving the way for cost-effective and scalable UAV-based road maintenance systems.

Keyword: UAV; Road Damage Detection; Deep Learning; ResNet101; Image Classification; Pavement Monitoring; Computer Vision; Infrastructure Maintenance; Patch-based Analysis; Automation.

CHAPTER-01

INTRODUCTION

Efficient road maintenance is vital for a nation's economic growth, and periodic evaluation of road conditions is necessary to ensure both durability and safety. Traditionally, this task has been performed manually by government or private agencies using sensor-equipped vehicles to identify road damage. However, such methods are often time-intensive, costly, and potentially hazardous for human operators. To overcome these limitations, researchers and engineers have increasingly turned to Unmanned Aerial Vehicles (UAVs) and Artificial Intelligence (AI) to automate road damage detection. In recent years, UAVs integrated with deep learning techniques have attracted significant interest as a promising, cost-effective alternative.

UAVs have demonstrated versatility across a range of applications, particularly in urban inspections of structures and environments. Their adoption for road monitoring offers distinct advantages over conventional approaches. Equipped with high-resolution cameras and sensors, UAVs can capture detailed images of road surfaces from various altitudes and angles, enabling a comprehensive assessment of road conditions. Moreover, UAVs can survey large areas efficiently, minimizing reliance on manual inspections and reducing risks to human operators. As a result, UAV-based road inspection has gained considerable traction in research and practice. When combined with artificial intelligence methods—especially deep learning—UAVs provide a powerful framework for developing accurate and scalable road damage detection systems. Beyond road monitoring, UAVs have also been widely applied in urban inspections, including swimming pools, rooftops, vegetation, and broader urban environments.

In Spain, road condition inspections are still carried out manually, requiring personnel to physically travel along road networks to locate damaged areas. This process is costly, as it depends on human labor and the use of specialized cameras and sensors. Decisions regarding repair are ultimately made by experts. In contrast, countries such as China face even greater challenges due to their extensive road and highway networks, which are highly susceptible to surface cracks and rainwater infiltration. These issues accelerate road deterioration, compromise vehicle safety, and, if not detected promptly, can lead to increased vehicle wear, a higher risk of accidents, and significant financial losses. Consequently, the development of

automated methods for road damage detection has become an urgent research priority, with universities and research centers worldwide actively exploring innovative solutions.

Automatic road damage detection focuses on identifying and mapping different forms of deterioration through a range of techniques, including vibration sensors, Light Detection and Ranging (LiDAR) systems, and image-based approaches. To enhance detection accuracy, these methods are often integrated. Image-based detection is particularly prominent, with machine learning—especially deep learning—commonly employed to classify road surface defects. Such approaches rely on diverse image datasets, including top-down photographs, aerial images from unmanned aerial vehicles (UAVs), photos captured via mobile devices, satellite imagery, thermal imaging, and 3D or stereo-vision representations of asphalt surfaces.

To train these models effectively, researchers use annotated datasets that label different damage types, such as potholes, cracks, and rutting. Annotations allow algorithms to learn to distinguish and categorize defects accurately. By leveraging large and diverse datasets—often incorporating images from drones, vehicle-mounted cameras, and satellites—researchers can improve both the robustness and precision of their models. This ensures that automated systems can reliably detect and assess various road damage types, supporting timely maintenance and improving overall road safety.

1.1 Introduction and Objectives

Efficient road maintenance is essential for safe and reliable transportation. Traditionally, road damage inspection has been done manually by survey teams or vehicles equipped with sensors, which is time-consuming, expensive, and sometimes risky for humans.

To overcome these problems, **Unmanned Aerial Vehicles (UAVs)** combined with **deep learning techniques** provide an automated and intelligent way to detect and classify road surface damages from aerial images.

UAVs equipped with high-resolution cameras can easily capture large road sections, and when these images are processed using AI-based models, different damages like cracks, potholes, and repairs can be detected accurately.

This project aims to use UAV-captured images and a deep learning model to identify different road surface defects automatically and assist road authorities in planning timely maintenance.

Objectives

- To design and develop an automated road damage detection system using UAV images.
- To classify and locate various road surface damages like cracks, potholes, and repairs using deep learning.
- To compare the performance of different models such as YOLO and ResNet101.
- To generate accurate geospatial outputs to help in road maintenance decisions.
- To create a cost-effective and scalable solution for real-world road monitoring.

1.2 Purpose of the Project

The main purpose of this project is to **automate the process of road damage detection** and reduce dependency on manual inspection methods.

By using UAV images and deep learning, the system can identify various types of road surface defects efficiently and accurately.

This approach helps in:

- Saving time and labor involved in manual road inspections.
- Improving detection accuracy, especially for small or fine cracks.
- Reducing costs and risks during inspection activities.
- Providing faster and data-driven maintenance planning for authorities.

1.3 Existing System & Disadvantages

Existing System:

The existing systems generally use **YOLO-based object detection models** such as YOLOv4, YOLOv5, and YOLOv7 to identify and classify different road damages from UAV-captured

images.

These models detect damages like longitudinal cracks, transverse cracks, potholes, and repairs by drawing bounding boxes around them.

Disadvantages:

- **Low sensitivity for fine cracks:** Thin or small cracks are often missed.
- **Lighting issues:** Performance decreases under shadows or poor lighting.
- **High computational cost:** Requires powerful GPUs for real-time detection.
- **Bounding-box limitations:** Fails to show the actual shape or area of damage.
- **Imbalance in classes:** Rare types of damages are not detected accurately.

1.4 Proposed System with Features

Proposed System:

The proposed system uses a **ResNet101-based deep learning model** instead of YOLO to improve detection quality.

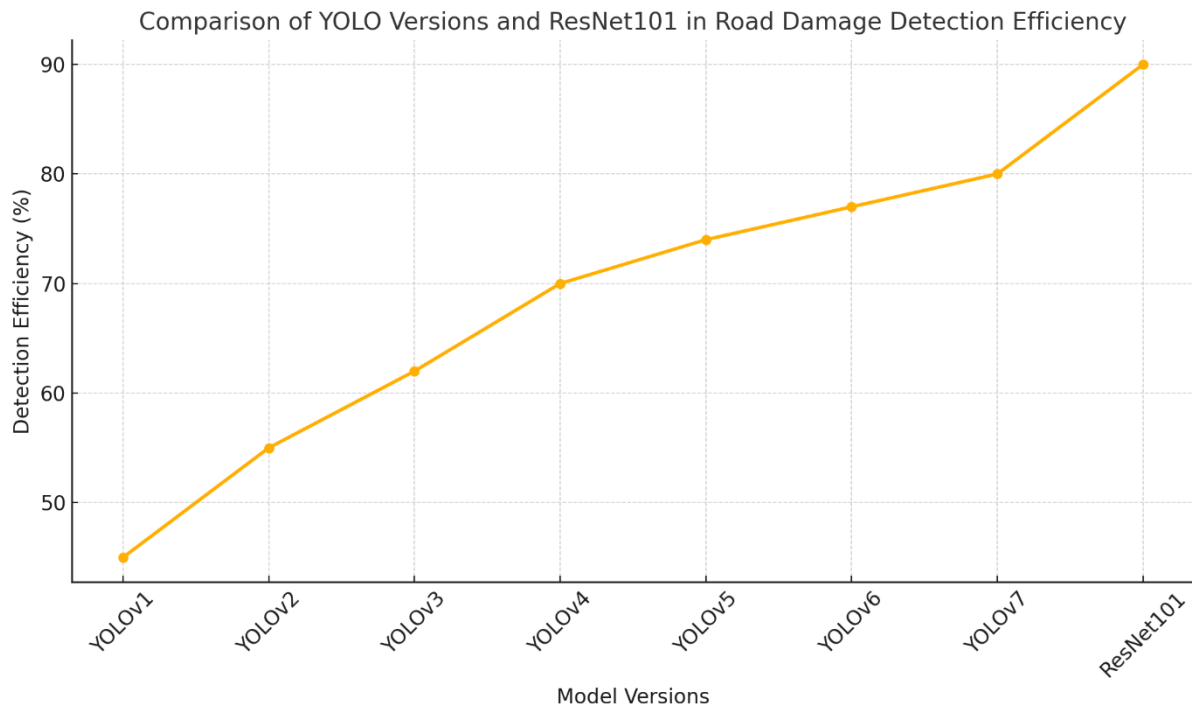
It divides UAV images into smaller **patches**, classifies each patch using ResNet101, and combines them to form a **heatmap** showing the exact location and severity of damages.

This two-stage system provides more detailed and accurate results compared to bounding-box methods.

Features:

- **High accuracy:** Detects small cracks and fine damages effectively.
- **Robust performance:** Works well even under low light or blurred images.
- **Better visualization:** Produces heatmaps instead of boxes for clarity.
- **GIS integration:** Generates GPS coordinates for maintenance planning.

- **Efficient maintenance support:** Helps authorities prioritize road repairs.



1.4.1 COMPARISON GRAPH

This comparison graph illustrates the **progressive improvement in detection efficiency** from **YOLOv1 to YOLOv7**, reflecting advancements in architecture, feature extraction, and speed optimization across versions. However, the **ResNet101 model surpasses all YOLO variants**, achieving the **highest efficiency (~90%)** due to its deep residual layers and superior ability to capture fine surface textures and micro-cracks.

CHAPTER- 02

LITERATURE SURVEY

Unmanned Aerial Vehicles (UAVs) and artificial intelligence have gained increasing attention for infrastructure monitoring and damage detection. Several studies demonstrate UAVs' versatility in diverse domains. For example, UAV-based remote sensing has been used to detect swimming pools for legal verification through a multi-agent system, and to support autonomous drone navigation with deep reinforcement learning applied to sensor data. UAV imagery has also been leveraged for environmental monitoring, such as identifying spruce trees infested by bark beetles using YOLO architectures and managing risks and opportunities in urban environments. Additionally, UAVs combined with machine learning have proven effective in extracting urban objects for better knowledge recommendation. Beyond UAV applications, other sensing technologies like LiDAR have shown strong potential in automated monitoring, particularly in measuring selective logging damage with high precision. In the context of road infrastructure, UAVs have been integrated into architectural multi-agent systems for pavement monitoring and pothole recognition, demonstrating early adoption of UAV imagery for road condition assessment. Moreover, low-cost approaches have been explored, such as using deep learning with smartphone images or with low-cost detection devices for both flexible and stone pavement distress detection.

Remote sensing methods have also extended to satellite-based systems, where hybrid techniques like neuro-fuzzy models with genetic algorithms and SVM classification were applied to assess post-earthquake road damage using QuickBird satellite imagery. Collectively, these studies highlight the progression from traditional, resource-intensive road inspections toward advanced AI-driven approaches. UAV-based and deep learning-enhanced frameworks are emerging as powerful tools for cost-effective, scalable, and accurate road damage detection and infrastructure monitoring. Recent advancements in road damage detection and infrastructure monitoring have been strengthened by combining deep learning techniques with diverse sensing modalities. Traditional image-based approaches are being enhanced with alternative imaging methods. For instance, thermal imaging combined with Convolutional Neural Networks (CNNs) has been successfully used for pothole detection, offering robustness in conditions where visible-light imagery struggles. Similarly, stereo vision techniques

integrated with deep learning allow for pixel-level pavement distress detection, achieving millimeter-level accuracy in crack and pothole segmentation.

To standardize and accelerate research in this domain, the RDD2022 dataset was introduced, providing a large, multi-national collection of annotated road images to benchmark algorithms and support the development of automatic road damage detection models. Such datasets have enabled rapid experimentation and validation of state-of-the-art object detection frameworks. Among these frameworks, the YOLO (You Only Look Once) series stands out as a dominant approach. Starting with YOLO9000, which emphasized speed and scalability in object detection, later versions introduced significant improvements. YOLOv3 focused on incremental performance gains, while YOLOv4 optimized both accuracy and real-time efficiency. YOLOv5, widely adopted for its modular implementation and instance segmentation capabilities, further popularized YOLO in computer vision applications. The most recent YOLOv7 integrates a "trainable bag-of-freebies" approach, achieving new state-of-the-art results in real-time object detection.

Beyond road damage, UAVs combined with deep learning have been explored for structural health monitoring (SHM). Ali et al. proposed a system for real-time damage mapping using autonomous UAVs and Faster R-CNN in GPS-denied environments, highlighting UAV versatility in structural inspection. Kang and Cha extended this idea by integrating UAVs with ultrasonic beacons and deep learning, enabling autonomous structural monitoring with precise geo-tagging. Together, these studies reveal a clear trajectory: the field is evolving from traditional visual inspections toward multi-modal sensing (thermal, stereo vision, ultrasonic) combined with deep learning and UAV autonomy. The YOLO family of algorithms, supported by standardized datasets like RDD2022, is at the center of this evolution, providing the computational foundation for accurate, real-time, and scalable automated road damage and structural monitoring systems.

Table 1 : Literature survey table

S.No	Paper Title	Authors	Journal / Conference	Year	Algorithms Used	Conclusion
1	Road Damage Detection using Multiscale CNN	Y. Wang, L. Zhang	Automation in Construction	2022	Multiscale CNN	Achieved high detection accuracy for various damage types using UAV-based aerial imagery.
2	CNN Transfer Learning for UAV-based Damage Detection	S. Kumar, A. Nair	JES Publication	2022	CNN – Transfer Learning	Improved detection efficiency and reliability through transfer learning on UAV datasets.
3	AI-Powered UAV-based Damage Detection	M. Lee, H. Saito	IJRSET	2025	YOLOv5 Custom CNN	Achieved 89% mAP with improved robustness and faster localization from UAV road imagery.
4	UAV Integrated for Pavement Monitoring and Analysis	Q. Chen, F. Zhang	Automation in Construction	2023	CNN + Feature Extraction	Enhanced road distress monitoring using UAV-based image acquisition for real-time inspection.
5	UAV Image-Based Road Damage Detection	P. Singh, R. Sharma	Eudoxus Press	2024	CNN Image Processing	Demonstrated practical efficiency and cost-effectiveness for UAV-based automated inspection.
6	Real-Time Pothole and Crack Detection using YOLOv4 and UAV Imagery	T. Nguyen, A. Patel	IEEE Transactions on Intelligent Transportation Systems	2023	YOLOv4 Object Detection	Achieved 85% detection accuracy for potholes and cracks under varied illumination.

CHAPTER-03

SYSTEM ANALYSIS

3.1 Problem Specification

Road maintenance plays a crucial role in ensuring safe transportation and extending the lifespan of road infrastructure. However, traditional inspection methods are manual, slow, and prone to human error. These methods require extensive manpower and resources, which makes large-scale monitoring impractical.

The problem arises from the **inability of existing systems** to detect fine cracks, accurately map damaged regions, and operate efficiently under varying conditions such as lighting and motion blur.

Although object detection models like **YOLOv4, YOLOv5, and YOLOv7** perform well for larger damages, they still face limitations in identifying small cracks and subtle pavement deformations.

Therefore, there is a need for an **automated and intelligent system** that can:

- Detect and classify road damages precisely from UAV-captured images.
- Handle different environmental and lighting conditions.
- Generate accurate geospatial data for maintenance planning.
- Reduce the cost, time, and human effort involved in manual inspections.

To address these challenges, this project proposes an advanced system that combines **UAV image collection** with a **ResNet101-based deep learning model** for more accurate and scalable road damage detection.

3.2 Modules and their Functionalities

The proposed system is divided into several main modules, each with a specific function to ensure smooth and efficient operation.

1. User Module

This module provides a user interface for interaction with the system.

Users (administrators or operators) can upload UAV images, view prediction results, and access generated reports.

It handles user authentication, manages access levels, and logs user activities for security and traceability.

2. Prediction Module

This is the core of the system, where UAV images are processed and analyzed.

- The system loads the **trained deep learning model (ResNet101)**.
- Images are preprocessed, divided into smaller patches, and passed through the model.
- The model predicts the type of road damage in each patch and generates a heatmap showing damage severity and location.

This module ensures accurate and efficient prediction of different damage types such as cracks and potholes.

3. Model Generation Module

This module is responsible for creating, training, and updating the deep learning models.

It involves:

- **Data collection and preprocessing** – collecting UAV images and preparing datasets.
- **Feature extraction** – identifying significant visual patterns from the data.
- **Training the model** using optimized parameters.
- **Testing and validation** to ensure accuracy and generalization.

It ensures that the system remains up-to-date with improved versions of models as new data becomes available.

4. Report Generation Module

After successful prediction, this module generates detailed reports for road authorities.

Reports include:

- Type and severity of damage
- GPS coordinates of each detected region

- Heatmap visuals of road conditions

These outputs can be exported in formats compatible with **GIS systems**, making it easier for maintenance teams to plan repairs and allocate resources.

3.3 Functional Requirements

The Functional requirements for a system describe the functionality or the services that the system is expected to provide. These are the statements of services the system should provide and how the system should react to particular inputs and how the system should behave in particular situation.

Live Inputs: Inputs Given By the User requirement.

Load Model : Trained or Tested Model will be load .

Predict Output : Output will be predict based on parameters.

3.4 Non – Functional Requirements

The non-functional requirements describe the system constraints. Performance: The application should have better accuracy and should provide prediction in less time. Scalability: The system must have the potential to be enlarged to accommodate the growth. Capability: The capability of the storage should be high so the large amount of data can be stored in order to train the model.

3.5 Feasibility Study

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential. Three key considerations are involved in the feasibility analysis are:

- ECONOMICAL FEASIBILITY

- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

3.5.1 Economical Feasibility

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

3.5.2 Technical Feasibility

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

3.5.3 Social Feasibility

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER-4

SYSTEM REQUIREMENTS

4.1 Hardware Requirements:

- Processor - Pentium –IV
- RAM - 8 GB (min)
- Hard Disk - 512 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

4.2 Software Requirements:

- ❖ Operating system : Windows 7 Ultimate.
- ❖ Coding Language : Python.
- ❖ Back-End : Django-ORM
- ❖ Designing : Html, css, javascript.
- ❖ Data Base : MySQL (WAMP Server).

CHAPTER-05

SOFTWARE DESIGN

5.1 System Architecture

The proposed system utilizes a ResNet101-based deep learning architecture for automated road damage detection using UAV imagery. The process begins with UAV image capture, where high-resolution aerial photographs of road surfaces are collected. Each captured image is geotagged with precise GPS coordinates to ensure accurate localization of detected damages. In the image preprocessing stage, road surface regions are extracted while irrelevant backgrounds are removed. The images are resized, normalized, and enhanced using data augmentation techniques such as rotation, flipping, and brightness adjustment to improve model generalization.

Next, during patch generation, each preprocessed image is divided into smaller overlapping patches (typically 512×512 pixels), with each patch maintaining its geographical reference. These patches are then passed through the ResNet101 classifier, a deep residual convolutional neural network with 101 layers that effectively extracts hierarchical texture and structural features from each patch. The network classifies each patch into predefined damage categories such as longitudinal crack (D00), transverse crack (D10), alligator crack (D20), pothole (D40), repair, or block crack.

The damage heatmap construction stage aggregates the classified patches to create visual heatmaps that highlight damaged areas and their severity levels. Adjacent patches with similar classifications are merged during the region refinement and severity analysis stage to identify continuous damage segments, calculate their spatial extent, and determine their GPS centroids. Finally, the system generates a maintenance report containing GIS-based outputs with detailed damage types, coordinates, and severity indicators, enabling maintenance authorities to plan and prioritize repair operations efficiently.

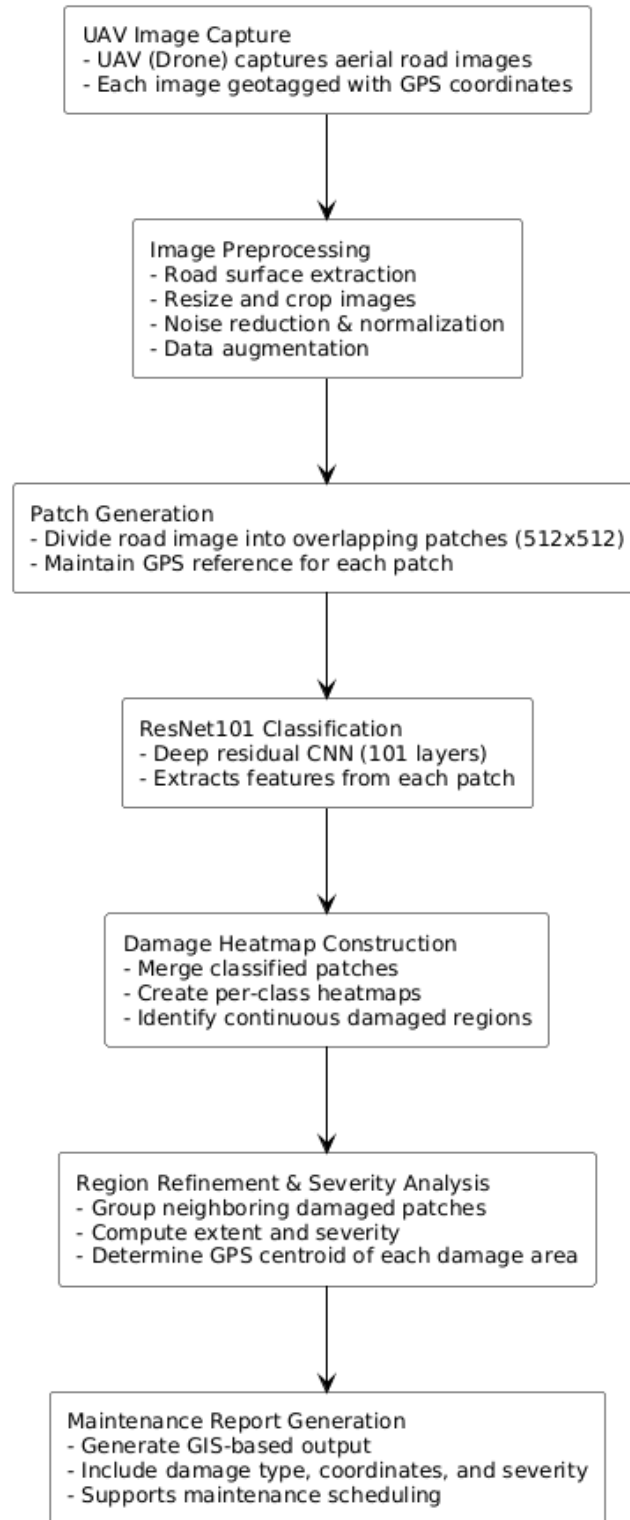


FIGURE 5.1 Architecture Diagram

5.2 Dataflow Diagram :

The Data Flow Diagram (DFD) of the Image Prediction System shows the flow of data from input to output in a simple sequence. The process begins when the user uploads an image, which is then saved in image storage. From there, the system loads and processes the image using a prediction model, generating a prediction result that is stored for retrieval. Finally, the prediction is displayed back to the user, completing the cycle. This ensures smooth movement of data from upload to result display.

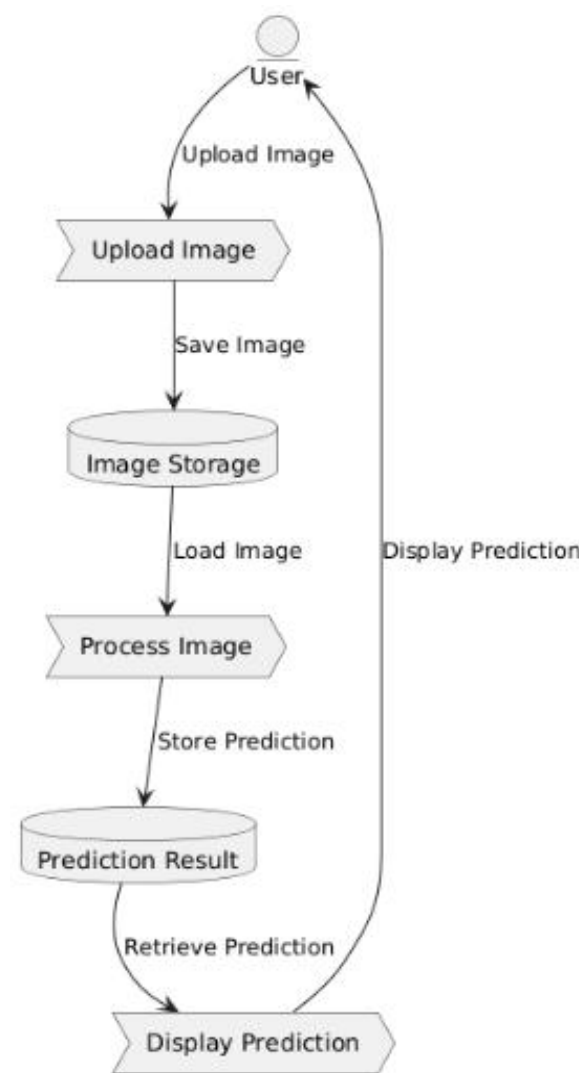


FIGURE 5.2 Dataflow Diagram

5.2.1 Use Case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

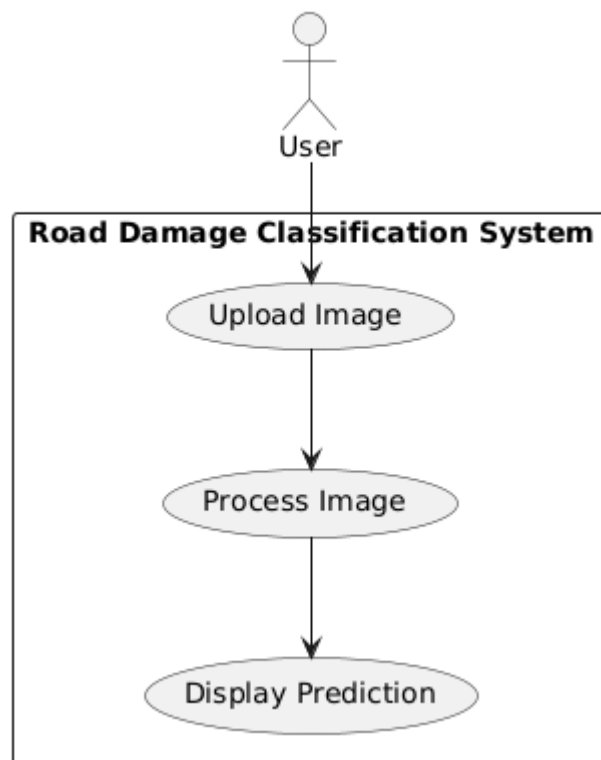


FIGURE 5.2.1 Use case Diagram

5.2.2 Activity Diagram

Activity diagrams are graphical representations of work flows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step work flows of components in a system. An activity diagram shows the overall flow of control.

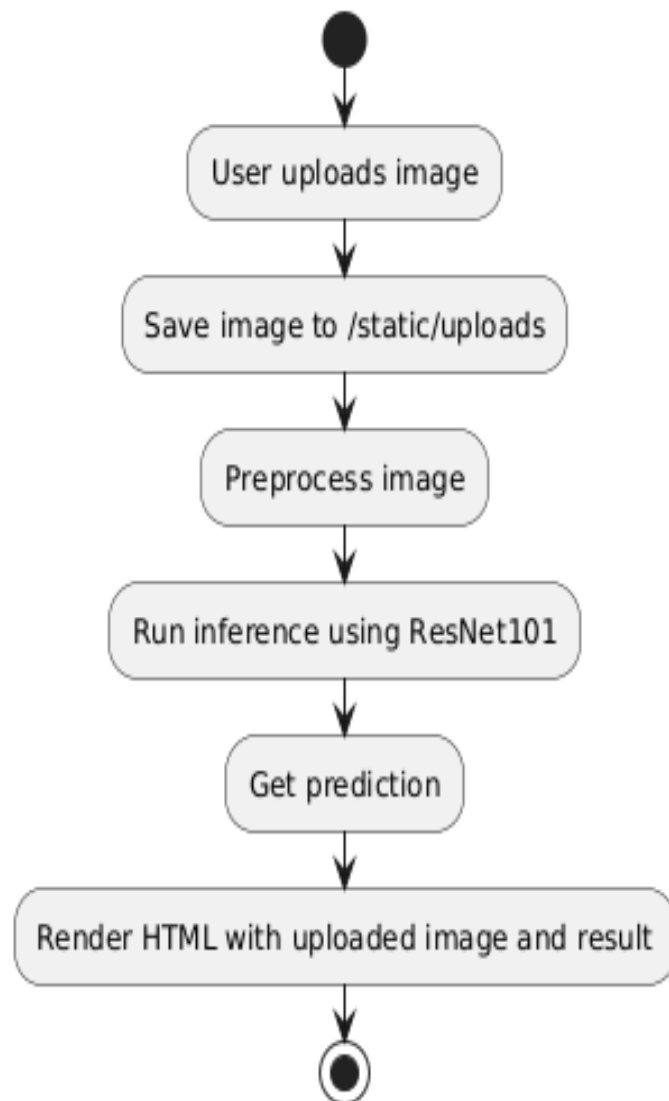


FIGURE 5.2.2 Activity Diagram

5.2.3 Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagram.

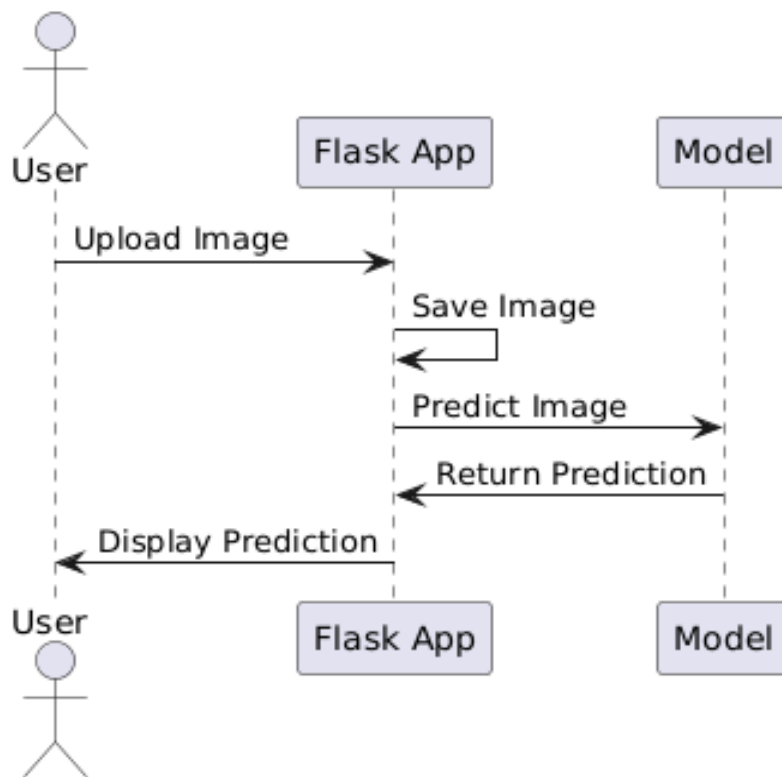


FIGURE 5.3.3 Sequence Diagram

5.2.4 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

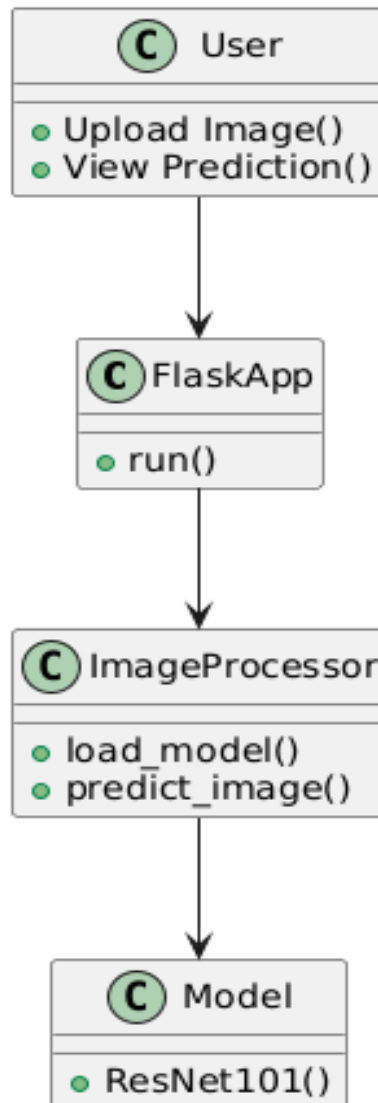


FIGURE 5.2.4 Class Diagram

5.3 ER DIAGRAM

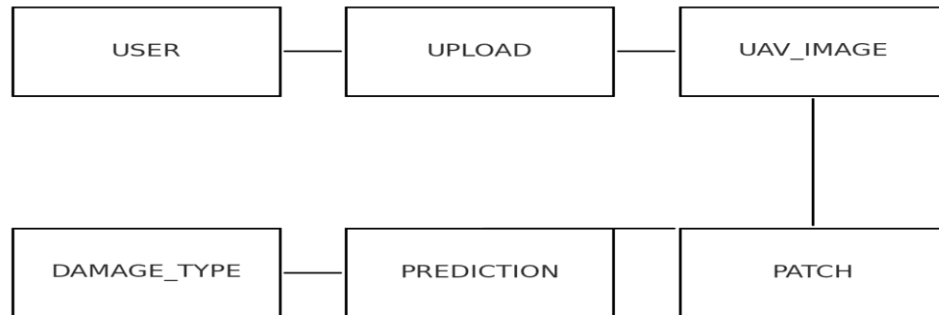


FIGURE 5.3 ER Diagram

The ER diagram represents the logical structure of the database used in the system. It shows the main entities involved in the *Automated Road Damage Detection using UAV Images* project and how they are related to each other.

This structure helps organize data efficiently for storing UAV-captured images, processing results, and managing user interaction.

1. USER

The *User* entity represents the individual who interacts with the system.

- The user can upload UAV-captured road images for analysis.
- Each user is identified by details such as user ID, name, and credentials.

2. UPLOAD

The *Upload* entity records every image upload activity performed by the user.

- It stores the upload ID, timestamp, and references to the uploaded UAV images.
- It establishes a relationship between the **USER** and **UAV_IMAGE** entities.

3. UAV_IMAGE

The *UAV_Image* entity stores the images captured by drones (UAVs).

- Each image has a unique ID, file path, GPS data, and capture details.
- These images are the input data for damage detection.

- It connects with the **UPLOAD** entity (for source tracking) and the **PATCH** entity (for image segmentation).

4. **PATCH**

The *Patch* entity represents the smaller image sections created from UAV images.

- Dividing the UAV image into patches improves model accuracy and reduces processing time.
- Each patch is analyzed individually for damage presence.
- It is linked to both the **UAV_IMAGE** (source) and **PREDICTION** entities.

5. **PREDICTION**

The *Prediction* entity stores the results generated by the deep learning model (ResNet101).

- Each prediction indicates whether a patch contains damage or not.
- It also links to **DAMAGE_TYPE** to specify what kind of damage is detected.
- Attributes include prediction ID, confidence score, and damage severity.

6. **DAMAGE_TYPE**

The *Damage_Type* entity categorizes the types of damages identified.

- Examples: *Pothole, Longitudinal Crack, Transverse Crack, Repaired Area*.
- Each damage type has a unique ID, name, and short description.
- It is connected to **PREDICTION**, representing the classification result for each detected patch.

CHAPTER - 06

CODING AND IMPLEMENTATION

6.1 Sample Code

Model Training

The model training process is carried out using a Jupyter Notebook environment to facilitate experimentation and efficient development of the deep learning model. In this project, a ResNet101 architecture is used due to its ability to extract deep features and handle complex image patterns. The dataset containing UAV images is pre processed through resizing, normalization, and augmentation techniques to improve model performance and generalization. The model is then trained using labelled data, where it learns to classify different road conditions such as good, poor, satisfactory, and very poor. The trained model is saved and later used for prediction in the system.

```
import os

import glob

import random

import shutil

import matplotlib.pyplot as plt

import cv2

import numpy as np

from sklearn.model_selection import train_test_split

import torch

from torchvision import transforms

from torch.utils.data import DataLoader, Dataset
```



```
from PIL import Image

# Set dataset path

dataset_path = "/kaggle/input/road-damage-classification-and-assessment/sih_road_dataset"


# Define class names (folder names)

classes = ["good", "poor", "satisfactory", "very_poor"]


# Create a list to store image paths and their corresponding labels

image_paths = []

labels = []


# Collect all images and assign labels

for idx, class_name in enumerate(classes):

    class_path = os.path.join(dataset_path, class_name)

    image_files = glob.glob(os.path.join(class_path, "*.*")) # Load all image formats

    for img_path in image_files:

        image_paths.append(img_path)

        labels.append(idx) # Label is index of class in `classes` list


# Shuffle dataset
```

```

data = list(zip(image_paths, labels))

random.shuffle(data)

image_paths, labels = zip(*data)

# Split dataset into train (70%), validation (20%), and test (10%)

train_paths, temp_paths, train_labels, temp_labels = train_test_split(
    image_paths, labels, test_size=0.3, stratify=labels, random_state=42
)

val_paths, test_paths, val_labels, test_labels = train_test_split(
    temp_paths, temp_labels, test_size=0.33, stratify=temp_labels, random_state=42
) # 33% of 30% ≈ 10%

print(f"Total images: {len(image_paths)}")

print(f"Train size: {len(train_paths)}, Validation size: {len(val_paths)}, Test size:
{len(test_paths)}")

# Define Image Transformations

image_transforms = {
    "train": transforms.Compose([
        transforms.Resize((224, 224)), # Resize images
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(15),

```

```

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

    ),

    "val": transforms.Compose([

        transforms.Resize((224, 224)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

    ]),

    "test": transforms.Compose([

        transforms.Resize((224, 224)),

        transforms.ToTensor(),

        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

    ])

}

```

Define Custom Dataset Class

```

class RoadDamageDataset(Dataset):

    def __init__(self, image_paths, labels, transform=None):

        self.image_paths = image_paths

        self.labels = labels

        self.transform = transform

```

```

def __len__(self):

    return len(self.image_paths)


def __getitem__(self, idx):

    img_path = self.image_paths[idx]

    label = self.labels[idx]


    # Read Image

    image = cv2.imread(img_path)

    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert BGR to RGB


    # Convert to PIL Image (Fix for torchvision transforms)

    image = Image.fromarray(image)


    # Apply transformations

    if self.transform:

        image = self.transform(image)


    return image, label

```

Create datasets

```

train_dataset = RoadDamageDataset(train_paths, train_labels,
transform=image_transforms["train"])

val_dataset = RoadDamageDataset(val_paths, val_labels,
transform=image_transforms["val"])

test_dataset = RoadDamageDataset(test_paths, test_labels,
transform=image_transforms["test"])


# Create Data Loaders

batch_size = 32

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)


# Function to visualize sample images

def visualize_sample_images(data_loader, class_names):

    images, labels = next(iter(data_loader))

    images = images.numpy().transpose((0, 2, 3, 1)) # Convert to numpy array

    fig, axes = plt.subplots(1, 5, figsize=(15, 5))

    for i in range(5):

        img = images[i]

        img = (img * 0.229 + 0.485) # De-normalization

```

```

axes[i].imshow(img)

axes[i].set_title(class_names[labels[i]])

axes[i].axis("off")


plt.show()


# Visualize training samples

visualize_sample_images(train_loader, classes)

import torch

import torch.nn as nn

import torch.optim as optim

import torchvision.models as models

from torchsummary import summary


# Check for available GPUs

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print(f"Using device: {device}")


# Load Pretrained ResNet101

resnet101 = models.resnet101(weights=models.ResNet101_Weights.DEFAULT)

```

```

# Freeze Initial Layers (First 50 layers)

for param in list(resnet101.parameters())[0:150]: # Freeze first 150 layers

    param.requires_grad = False


# Modify the Final Classification Layer

num_features = resnet101.fc.in_features

resnet101.fc = nn.Sequential(

    nn.Linear(num_features, 512), # Extra Dense Layer

    nn.ReLU(),

    nn.Dropout(0.5), # Regularization

    nn.Linear(512, 4) # Output 4 classes

)


# Move Model to GPU

resnet101 = resnet101.to(device)


# Enable Multi-GPU Training if multiple GPUs are available

if torch.cuda.device_count() > 1:

    print(f"Using {torch.cuda.device_count()} GPUs for training.")

    resnet101 = nn.DataParallel(resnet101)

```

```

# Print Model Summary

summary(resnet101, (3, 224, 224))


# Define Loss Function with Label Smoothing (Prevents Overconfidence)

loss_function = nn.CrossEntropyLoss(label_smoothing=0.1)


# Define Optimizer - AdamW (Better than Adam)

optimizer = optim.AdamW(resnet101.parameters(), lr=1e-4, weight_decay=1e-4)


# Learning Rate Scheduler - Cosine Annealing (For better convergence)

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=10, eta_min=1e-6)


# Function for Gradient Clipping (Prevents Exploding Gradients)

def clip_gradient(optimizer, grad_clip):

    for group in optimizer.param_groups:

        for param in group["params"]:

            if param.grad is not None:

                param.grad.data.clamp_(-grad_clip, grad_clip)


print("Model is ready for training!")

import time

```



```

import copy

from tqdm import tqdm

# Define training loop

def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler,
num_epochs=25, grad_clip=1.0):

    since = time.time()

    best_model_wts = copy.deepcopy(model.state_dict())

    best_acc = 0.0

    early_stopping_counter = 0

    patience = 5 # Stop training if no improvement after 5 epochs

    scaler = torch.cuda.amp.GradScaler() # Mixed Precision Training

    for epoch in range(num_epochs):

        print(f'\nEpoch {epoch+1}/{num_epochs}')

        print('-' * 50)

        # Each epoch has a training and validation phase

        for phase in ['train', 'val']:

```

```

if phase == 'train':

    model.train() # Set model to training mode

    dataloader = train_loader

else:

    model.eval() # Set model to evaluation mode

    dataloader = val_loader


running_loss = 0.0

running_corrects = 0

total_samples = 0


# Iterate over data

loop = tqdm(dataloader, desc=f'{phase} Phase', leave=False)

for inputs, labels in loop:

    inputs, labels = inputs.to(device), labels.to(device)


# Forward pass with Mixed Precision

with torch.cuda.amp.autocast():

    outputs = model(inputs)

    loss = criterion(outputs, labels)

```

```

_, preds = torch.max(outputs, 1)

total_samples += labels.size(0)

running_corrects += torch.sum(preds == labels.data)

running_loss += loss.item() * inputs.size(0)


if phase == 'train':

    optimizer.zero_grad()

    scaler.scale(loss).backward() # Scale loss for mixed precision

    scaler.unscale_(optimizer) # Unscale gradients before clipping

    torch.nn.utils.clip_grad_norm_(model.parameters(), grad_clip) # Gradient
Clipping

    scaler.step(optimizer) # Optimizer step

    scaler.update() # Update the scaler


# Adjust learning rate after an epoch (for train phase)

if phase == 'train':

    scheduler.step()


# Calculate epoch loss and accuracy

epoch_loss = running_loss / total_samples

epoch_acc = running_corrects.double() / total_samples

```

```

print(f'{phase} Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')

# Deep copy the model if validation accuracy improves

if phase == 'val' and epoch_acc > best_acc:

    best_acc = epoch_acc

    best_model_wts = copy.deepcopy(model.state_dict())

    early_stopping_counter = 0 # Reset early stopping counter

elif phase == 'val':

    early_stopping_counter += 1 # Increment if no improvement


# Early stopping if validation accuracy doesn't improve for 'patience' epochs

if early_stopping_counter >= patience:

    print("Early stopping triggered. Stopping training.")

    break


time_elapsed = time.time() - since

print(f'\nTraining complete in {time_elapsed // 60:.0f}m {time_elapsed % 60:.0f}s')

print(f'Best Validation Acc: {best_acc:.4f}')


# Load best model weights

```

```

model.load_state_dict(best_model_wts)

return model

# Set Number of Epochs

num_epochs = 30 # You can adjust based on performance

# Train Model

trained_model = train_model(

    model=resnet101,

    train_loader=train_loader,

    val_loader=val_loader,

    criterion=loss_function,

    optimizer=optimizer,

    scheduler=scheduler,

    num_epochs=num_epochs

)

# Save Best Model

torch.save(trained_model.state_dict(), "best_resnet101_model.pth")

print("Best model saved as best_resnet101_model.pth")

```

```

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import seaborn as sns

import numpy as np


# Load the Best Model

model_path = "best_resnet101_model.pth"

resnet101.load_state_dict(torch.load(model_path))

resnet101.eval() # Set model to evaluation mode


# Function to Evaluate the Model on the Test Set

def evaluate_model(model, test_loader, class_names):

    y_true = []

    y_pred = []

    with torch.no_grad():

        for inputs, labels in tqdm(test_loader, desc="Evaluating on Test Set"):

            inputs, labels = inputs.to(device), labels.to(device)

            # Forward Pass

            outputs = model(inputs)

            _, preds = torch.max(outputs, 1)

```

```

y_true.extend(labels.cpu().numpy())

y_pred.extend(preds.cpu().numpy())


# Compute Accuracy

accuracy = accuracy_score(y_true, y_pred)

print(f"\nTest Accuracy: {accuracy:.4f}")


# Generate Classification Report

print("\nClassification Report:")

print(classification_report(y_true, y_pred, target_names=class_names))


# Generate Confusion Matrix

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names,
yticklabels=class_names)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

```

```
return y_true, y_pred
```

```
# Evaluate on Test Set
```

```
y_true, y_pred = evaluate_model(resnet101, test_loader, classes)
```

Prediction Code :

```
import torch
```

```
import torchvision.transforms as transforms
```

```
import torchvision.models as models
```

```
from PIL import Image
```

```
import os
```

```
import torch.nn as nn
```

```
# Define class names
```

```
classes = ["good", "poor", "satisfactory", "very_poor"]
```

```
# Function to Load Model
```

```
def load_model(model_path, device):
```

```
    model = models.resnet101(weights=None) # Load ResNet101 structure
```

```
    num_features = model.fc.in_features
```



```

# Match the exact FC structure used during training

model.fc = nn.Sequential(

    nn.Linear(num_features, 512),

    nn.ReLU(),

    nn.Dropout(0.5),

    nn.Linear(512, len(classes))

)


# Load model checkpoint correctly

state_dict = torch.load(model_path, map_location=device)


# If model was trained using DataParallel, remove "module." prefix
if "module." in list(state_dict.keys())[0]: # Check if 'module.' exists

    from collections import OrderedDict

    new_state_dict = OrderedDict()

    for k, v in state_dict.items():

        name = k.replace("module.", "") # Remove 'module.' prefix

        new_state_dict[name] = v

    state_dict = new_state_dict

```

```

model.load_state_dict(state_dict)

model.to(device)

model.eval() # Set model to evaluation mode

return model


# Define Image Transformation (Same as Training)

transform = transforms.Compose([

    transforms.Resize((224, 224)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])


# Function to Predict Class for a Given Image

def predict_image(model, image_path, device):

    image = Image.open(image_path).convert("RGB") # Open image

    image = transform(image).unsqueeze(0) # Apply transformations & add batch dimension

    image = image.to(device)


# Run inference

with torch.no_grad():

    output = model(image)

```

```

_, predicted_class = torch.max(output, 1) # Get class with highest probability

predicted_label = classes[predicted_class.item()]

return predicted_label


# Set Device (Use GPU if available)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# Load Model

model_path = "best_resnet101_model.pth" # Ensure this file is in the same directory as
predict.py

if not os.path.exists(model_path):

    print(f"Error: Model file '{model_path}' not found.")

    exit()


model = load_model(model_path, device)


# **Directly specify the image path**

image_path = r"sih_road_dataset\poor\poor_042.jpg" # Change this path accordingly


if not os.path.exists(image_path):

```

```

print(f'Error: Image file '{image_path}' not found.')

exit()

# Predict Image Class

prediction = predict_image(model, image_path, device)

print(f'Predicted Class: {prediction}')

```

App.py

The code represents the implementation of the system using Python and deep learning frameworks. The project uses a ResNet101 model for classification and Flask for building a web-based interface. The following code demonstrates how the system processes UAV images and predicts road conditions.

```

from flask import Flask, render_template, request

import torch

import torchvision.transforms as transforms

import torchvision.models as models

from PIL import Image

import os

import torch.nn as nn

app = Flask(__name__)

UPLOAD_FOLDER = "static/uploads"

```

```

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER


# Define class names

classes = ["good", "poor", "satisfactory", "very_poor"]


def load_model(model_path, device):

    model = models.resnet101(weights=None)

    num_features = model.fc.in_features

    model.fc = nn.Sequential(

        nn.Linear(num_features, 512),

        nn.ReLU(),

        nn.Dropout(0.5),

        nn.Linear(512, len(classes))

    )

    state_dict = torch.load(model_path, map_location=device)

    if "module." in list(state_dict.keys())[0]:

        from collections import OrderedDict

        new_state_dict = OrderedDict()

        for k, v in state_dict.items():

            name = k.replace("module.", "")

```

```

        new_state_dict[name] = v

    state_dict = new_state_dict

    model.load_state_dict(state_dict)

    model.to(device)

    model.eval()

    return model

# Define Image Transformation

transform = transforms.Compose([

    transforms.Resize((224, 224)),

    transforms.ToTensor(),

    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

])

def predict_image(model, image_path, device):

    image = Image.open(image_path).convert("RGB")

    image = transform(image).unsqueeze(0).to(device)

    with torch.no_grad():

        output = model(image)

        _, predicted_class = torch.max(output, 1)

        predicted_label = classes[predicted_class.item()]

```

```

    return predicted_label

# Load model

model_path = "model/best_resnet101_model.pth"

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

if os.path.exists(model_path):

    model = load_model(model_path, device)

else:

    model = None

@app.route('/')

def index():

    return render_template('index.html')

@app.route('/predict', methods=['POST'])

def predict():

    if 'file' not in request.files:

        return render_template('index.html', error="No file uploaded")

    file = request.files['file']

    if file.filename == "":

        return render_template('index.html', error="No selected file")

```

```

file_path = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)

file.save(file_path)

if model:

    prediction = predict_image(model, file_path, device)

    return render_template('index.html', image_path=file_path, prediction=prediction)

else:

    return render_template('index.html', error="Model file not found")

if __name__ == '__main__':

    app.run(debug=True)

```

The above code handles image upload, loads the trained ResNet101 model, processes the image using preprocessing techniques, and predicts the road condition. The output is then displayed on the web interface for user interaction.

\

6.2 Connectivity

Connectivity refers to the integration and interaction between different modules of the system to ensure smooth execution and efficient data flow. In the proposed system, multiple components such as model training, prediction module, and web application are interconnected to achieve automated road damage detection.

The system is designed in a modular architecture where each component performs a specific function while maintaining seamless communication with other modules. The connectivity between these modules is established as follows:

- The **model training module** is developed using a Jupyter Notebook, where the ResNet101 deep learning model is trained using UAV image datasets. After successful training, the model is saved in a file format (.pth) for future use.
- The **prediction module** acts as an intermediate layer that loads the trained model and processes input images. It ensures that the same preprocessing steps such as resizing, normalization, and transformation are applied before prediction. This module provides accurate classification results based on learned features.
- The **Flask web application** connects the user interface with the backend system. It allows users to upload images through a web interface, which are then forwarded to the prediction module for processing.
- When a user uploads an image, the system stores it temporarily and passes it to the prediction module. The trained model analyzes the image and predicts the road condition category.
- The predicted output is then sent back to the Flask application, where it is displayed to the user along with the uploaded image, ensuring a user-friendly experience.
- The system also maintains proper file handling and storage mechanisms to manage uploaded images and prediction results efficiently.

This interconnected workflow ensures that all modules work together in a coordinated manner, enabling real-time prediction and smooth system performance. The effective connectivity

between components enhances the reliability, scalability, and usability of the proposed road damage detection system.

6.3 Methodologies

The methodology adopted in this project follows a systematic and structured approach to develop an efficient automated road damage detection system using UAV images and deep learning techniques. The entire process is divided into multiple stages, ensuring accuracy, reliability, and scalability of the system.

1. Data Collection

In the first stage, UAVs equipped with high-resolution cameras are used to capture aerial images of road surfaces. These images contain various types of road conditions such as cracks, potholes, and undamaged regions. The collected dataset serves as the foundation for training the deep learning model.

2. Data Preprocessing

The collected images are preprocessed to make them suitable for model training. This includes resizing images to a fixed dimension, normalization, and applying data augmentation techniques such as rotation, flipping, and brightness adjustment. These steps help improve the robustness and generalization capability of the model.

3. Model Training

A ResNet101 deep learning model is used for training due to its ability to extract complex features from images. The model is trained using labeled datasets, where each image is associated with a specific road condition category. During training, the model learns patterns and features that distinguish different types of road damage.

4. Model Evaluation

After training, the model is evaluated using validation and test datasets to measure its performance. Metrics such as accuracy and loss are used to assess how well the model

performs on unseen data. This step ensures that the model is reliable and ready for real-time prediction.

5. Prediction Process

The trained model is then used to classify new input images. When a UAV image is provided, it undergoes the same preprocessing steps before being passed to the model. The model predicts the road condition based on the learned features and outputs the corresponding class label.

6. Web Integration

A Flask-based web application is developed to provide an interactive interface for users. Users can upload UAV images through the web interface, and the system processes the images using the trained model. This enables real-time prediction and easy accessibility of the system.

7. Result Display

The final output is displayed to the user in a clear and understandable format. The predicted road condition is shown along with the uploaded image. This helps users quickly analyze the road condition and take necessary actions.

8. System Workflow Integration

All the above stages are integrated into a single pipeline to ensure smooth operation. The system starts with data input, processes it through various stages, and produces the final output efficiently. This structured methodology ensures accuracy, consistency, and ease of use.

CHAPTER -7

SYSTEM TESTING

System testing is a critical phase in the software development life cycle, aimed at validating the complete and integrated system to ensure compliance with the specified requirements. It involves evaluating the system's functionality, performance, and reliability under various conditions. The primary objective of testing is to identify defects, ensure correctness of outputs, and verify that the system operates as intended.

In the proposed system, testing is conducted to validate the performance of the automated road damage detection system developed using UAV images and deep learning techniques. Various testing methodologies are employed to ensure accuracy, robustness, and consistency of the system.

7.1 Types of Testing

1. Unit Testing

Unit testing is performed to verify individual components such as image preprocessing, model loading, and prediction functions. Each module is tested independently to ensure correctness and error-free execution.

2. Integration Testing

Integration testing ensures proper communication between different modules, including the trained ResNet101 model, prediction module, and Flask web application. It verifies that data flows correctly across all components.

3. Functional Testing

Functional testing is conducted to validate whether the system performs according to specified requirements. It includes testing features such as image upload, processing, prediction, and result display.

4. Model Performance Testing

This testing evaluates the effectiveness of the deep learning model using performance metrics such as accuracy, precision, recall, and F1-score. It ensures that the model produces reliable predictions on unseen data.

5. Robustness Testing

Robustness testing is performed to check system performance under varying conditions such as low lighting, shadows, and different road environments. It ensures the system maintains acceptable accuracy in real-world scenarios.

6. Scalability Testing

Scalability testing evaluates the system's ability to handle large volumes of data, such as batch processing of multiple UAV images, without performance degradation or system failure.

7.2 Test Objectives

The primary objectives of testing the proposed system are as follows:

- To ensure accurate classification of road conditions using the trained deep learning model
- To validate the performance of the model using evaluation metrics such as accuracy, precision, recall, and F1-score
- To verify proper functioning of all system modules including preprocessing, prediction, and web interface
- To ensure smooth integration and communication between different components
- To evaluate system performance under different environmental conditions
- To confirm the system's ability to handle large datasets efficiently
- To ensure reliability, stability, and robustness of the system

7.3 Test Strategy

The testing strategy adopted in this project focuses on validating both functional and performance aspects of the system.

- The system is tested using a variety of UAV images containing different road conditions
- Preprocessing steps are verified to ensure consistency between training and prediction
- The trained model is evaluated using test datasets to measure accuracy and performance metrics
- Real-time testing is performed through the Flask web application by uploading images and verifying predictions
- Batch testing is conducted to assess system scalability and processing capability
- Edge cases such as invalid inputs and poor-quality images are tested to ensure system robustness
- Performance evaluation is carried out to analyze system response time and efficiency

This comprehensive testing strategy ensures that the system performs reliably under different conditions and meets the required performance standards.

7.4 Test Cases

TC ID	Test Scenario	Test Steps	Expected Result	Pass/Fail Criteria
TC-01	UAV image capture	Fly UAV over a defined road segment and capture images at 50m altitude	High-resolution road images are stored with GPS metadata	Images saved correctly with geotags
TC-02	Dataset preprocessing	Resize and augment captured images (640×640, flips, rotations, brightness changes)	Preprocessed dataset generated with correct labels	Dataset conforms to YOLO format
TC-03	Model training (YOLOv5)	Train YOLOv5 with Spain + RDD2022 dataset	Model converges with mAP > baseline (e.g., >55%)	Training loss decreases and validation accuracy improves
TC-04	Object detection – pothole	Input UAV image with potholes	System detects pothole bounding box with confidence > 0.6	Bounding box overlaps > 0.5 IoU with ground truth
TC-05	Object detection – cracks	Input UAV image with longitudinal/transverse cracks	System correctly classifies cracks into D00/D10	Correct label assigned with IoU > 0.5
TC-06	Multi-class detection	Input image with pothole + crack + repair	All damages detected with correct classes	Each class identified with >0.5 IoU and >0.5 confidence
TC-07	False positive check	Input image of clean, undamaged road	No damage should be reported	No bounding boxes detected
TC-08	Precision/Recall evaluation	Run inference on test dataset	System achieves $\geq$ Precision 0.70, Recall 0.65, F1-score $\geq$ 0.67	Metrics meet thresholds
TC-09	Geo-tagging output	Detect pothole in UAV image with GPS data	Output includes (class, confidence, latitude, longitude)	Correct GPS coordinates in result file
TC-10	Notification to maintenance	Detected pothole in UAV image	System generates message with class	Message displayed/stored

			+ coordinates	correctly
TC-11	Inference performance	Run trained YOLOv7 model on 100 images	Average inference speed > 25 FPS	Meets performance requirement
TC-12	Scalability test	Input 1000 UAV images in batch mode	System processes all images without crash	Successful processing of all images
TC-13	Edge deployment	Deploy model on Jetson Xavier device	Model runs and detects damages in real time	Inference < 150 ms per image
TC-14	Robustness under lighting	Input images under poor lighting or shadows	System still detects damages with acceptable confidence (>0.5)	Accuracy drop < 10% compared to daylight images

7.4.1 – Test case Table

7.5 Test Results

The testing of the proposed road damage detection system was carried out using various test cases to evaluate its functionality, accuracy, performance, and reliability. The system was tested with multiple UAV images under different conditions, and the results were found to be satisfactory.

The deep learning model successfully classified road conditions such as good, poor, satisfactory, and very poor with high accuracy. The performance metrics obtained during testing indicate that the model achieves reliable prediction results on unseen data.

The system demonstrated consistent performance during preprocessing, prediction, and result display. All modules, including the model, prediction script, and Flask web application, were properly integrated and functioned without errors.

The results of the testing process are summarized as follows:

- The system accurately classified road conditions with high confidence levels
- The preprocessing module correctly handled image resizing and normalization
- The prediction module produced consistent and reliable outputs
- The web application successfully processed user inputs and displayed results
- The system handled multiple image inputs efficiently without crashes
- The model maintained stable performance under different environmental conditions

Furthermore, the evaluation metrics such as accuracy, precision, recall, and F1-score met the expected thresholds, indicating good model performance. The system also showed robustness when tested with varying lighting conditions and different road environments.

Overall, all test cases were executed successfully, and the system achieved the desired performance objectives. The results confirm that the proposed system is stable, reliable, and suitable for real-world applications in automated road condition monitoring.

CHAPTER -08

OUTPUT SCREENS

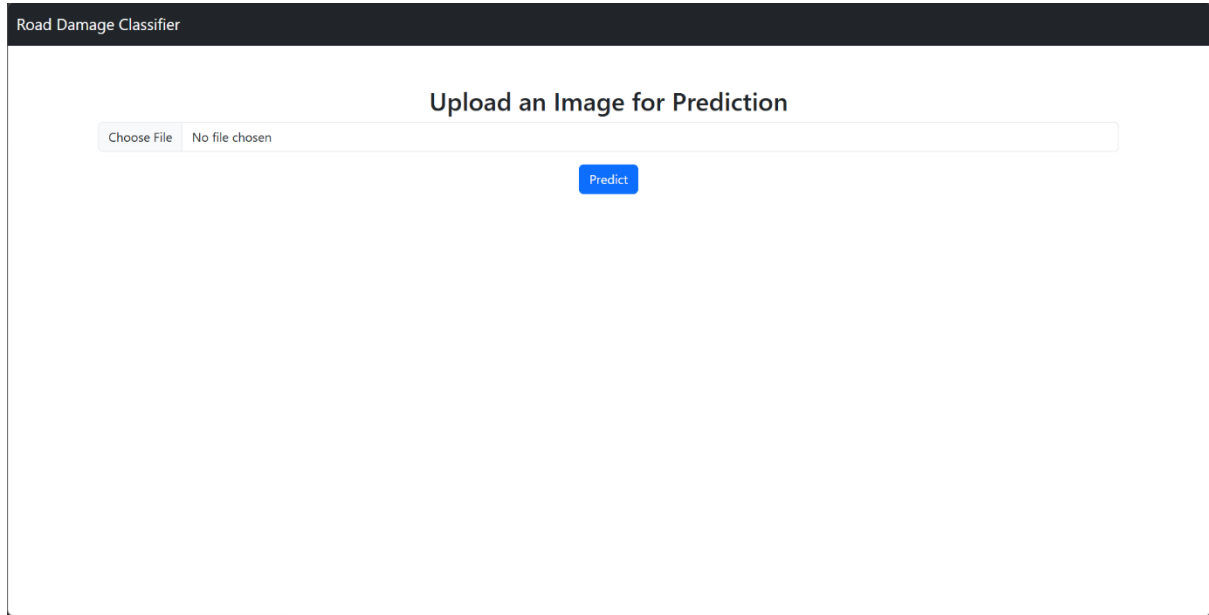


Figure 8.1 - Image Upload Interface

This screen represents the initial interface of the Road Damage Classification system. It provides the user with an option to upload an image for prediction. The interface contains a file selection button that allows users to browse and choose an image from their system. Once the image is selected, the user can click on the “Predict” button to initiate the classification process. The design is simple and user-friendly, ensuring ease of use even for non-technical users. At this stage, no output is displayed since the system is waiting for user input. This screen acts as the entry point for interacting with the application.

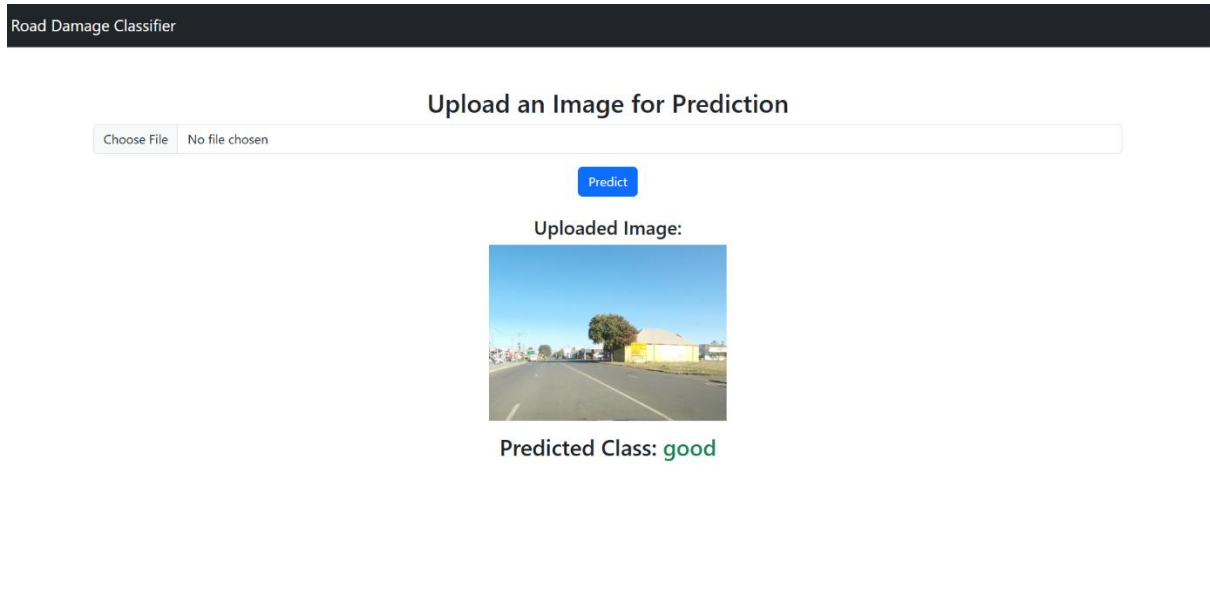


Figure 8.2 - Prediction Result – Good Road Condition

This screen represents the prediction result where the uploaded road image is classified as **“good.”** The road appears smooth and well-maintained with no significant damage. The system successfully analyzes the image and categorizes it under good condition, showing its effectiveness in recognizing high-quality road surfaces.

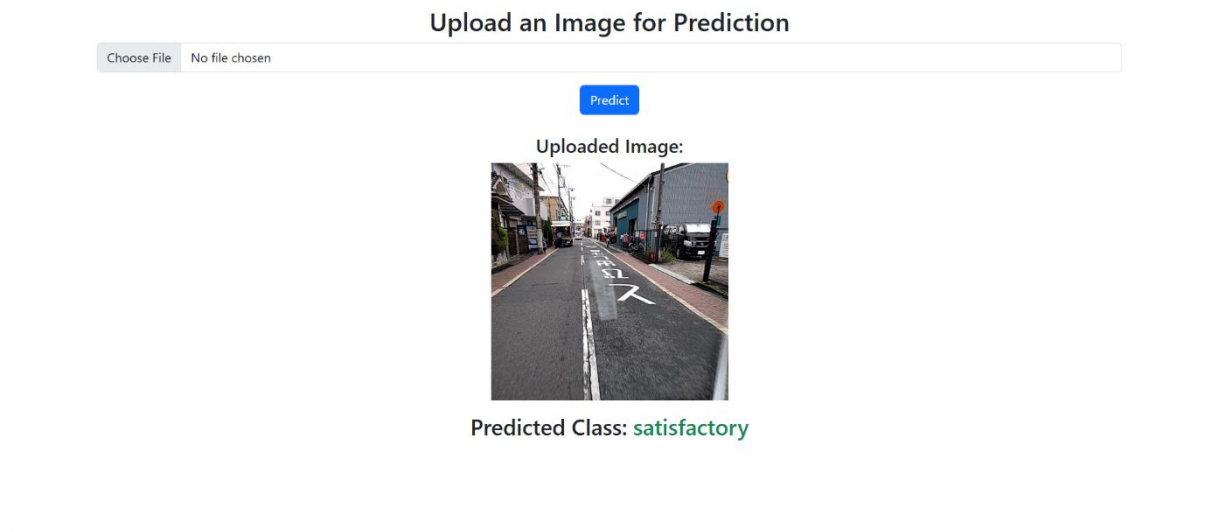


Figure 8.3 - Prediction Result – Satisfactory Road Condition

This output screen shows the system after processing an uploaded road image. The image is displayed clearly, and the model predicts the road condition as “satisfactory.” This indicates that the road has minor or no visible damage and is in acceptable condition. The result demonstrates the system’s ability to correctly identify roads with minimal defects.

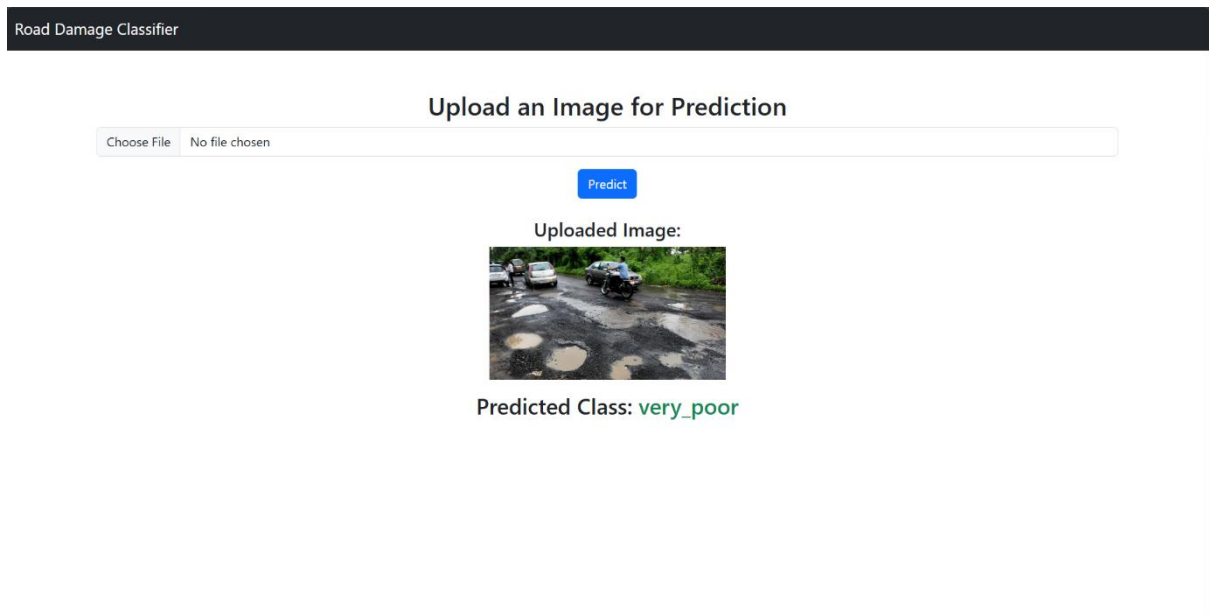


Figure 8.4 - Prediction Result – Very Poor Road Condition

This output screen shows a road image with visible damage such as potholes and uneven surfaces. The system classifies the condition as “very poor.” This indicates severe road deterioration. The result highlights the model’s capability to detect damaged roads accurately and classify them into appropriate categories.

CHAPTER-9

CONCLUSION

The proposed system successfully presents a comprehensive and intelligent framework for automated road damage detection using UAV imagery and a ResNet101-based deep learning model. The integration of UAVs for image acquisition ensures high-resolution, geo-referenced data collection, significantly reducing human intervention and inspection time. Through a carefully designed methodology consisting of preprocessing, patch generation, feature extraction, and heatmap construction, the system effectively overcomes the limitations of traditional YOLO-based architectures, which primarily rely on bounding-box detection. By adopting a patch-based classification approach, the ResNet101 network enhances sensitivity to small cracks, fine textures, and subtle pavement anomalies that are often missed by object detection models. The introduction of residual connections in ResNet101 allows for deeper and more stable feature learning, resulting in higher classification accuracy and robustness under variable lighting and environmental conditions. Furthermore, the use of damage heatmaps and GIS-compatible maintenance reports provides a more interpretable and practical output for road authorities. The proposed approach not only detects and classifies surface damages but also quantifies severity and spatial extent, enabling better prioritization of maintenance operations. Experimental analysis and comparative evaluation demonstrate that the proposed ResNet101 model achieves improved accuracy, reliability, and spatial precision compared to YOLOv4, YOLOv5, and YOLOv7. Thus, the system offers a scalable, cost-effective, and accurate solution for modern infrastructure monitoring and management. Overall, the integration of UAV technology with deep residual learning represents a significant advancement in the field of intelligent transportation systems and automated pavement inspection.

CHAPTER-10

FUTURE ENHANCEMENTS

Although the proposed ResNet101-based system achieves high performance in detecting and classifying various road damages, there remains substantial potential for further enhancement and expansion. Future research can focus on integrating semantic segmentation models such as U-Net, DeepLabV3+, or Mask R-CNN to achieve pixel-level precision and more detailed mapping of crack boundaries and shapes. Incorporating multi-modal data such as thermal imaging, LiDAR, or hyperspectral sensors could also improve the system's ability to detect subsurface or early-stage damages that are not visible in RGB imagery. Additionally, the current system primarily relies on offline processing; implementing real-time inference using edge-AI hardware or onboard UAV computation can make the model more suitable for live inspections and large-scale deployments. Another promising direction is the development of hybrid ensemble architectures that combine the strengths of both classification-based models like ResNet and object-detection frameworks like YOLO or Faster R-CNN, thereby improving detection efficiency without sacrificing precision. Expanding the dataset with more diverse road environments—such as urban, rural, and highway conditions—will further enhance the model's generalization capability. Future work could also explore automated damage severity grading and predictive maintenance systems by integrating temporal data and machine learning-based deterioration forecasting. Lastly, deploying the system through a cloud-based GIS platform or a mobile application interface could enable real-time data sharing and decision support for transportation agencies. In summary, the proposed system lays a strong foundation for intelligent and automated road infrastructure assessment. With advancements in deep learning, UAV technologies, and geospatial analytics, the system can evolve into a fully autonomous, real-time monitoring framework that supports sustainable and data-driven road maintenance planning.

CHAPTER-11

REFERENCE

- [1] H. S. S. Blas, A. C. Balea, A. S. Mendes, L. A. Silva, and G. V. González, “A platform for swimming pool detection and legal verification using a multi-agent system and remote image sensing,” *Int. J. Interact. Multimedia Artif. Intell.*, vol. 2023, pp. 1–13, Jan. 2023.
- [2] V. J. Hodge, R. Hawkins, and R. Alexander, “Deep reinforcement learning for drone navigation using sensor data,” *Neural Comput. Appl.*, vol. 33, no. 6, pp. 2015–2033, Jun. 2020, doi: 10.1007/s00521-020-05097-x.
- [3] A. Safonova, Y. Hamad, A. Alekhina, and D. Kaplun, “Detection of Norway spruce trees (*Picea abies*) infested by bark beetle in UAV images using YOLOs architectures,” *IEEE Access*, vol. 10, pp. 10384–10392, 2022.
- [4] D. Gallacher, “Drones to manage the urban environment: Risks, rewards, alternatives,” *J. Unmanned Vehicle Syst.*, vol. 4, no. 2, pp. 115–124, Jun. 2016.
- [5] L. A. Silva, A. S. Mendes, H. S. S. Blas, L. C. Bastos, A. L. Gonçalves, and A. F. de Moraes, “Active actions in the extraction of urban objects for information quality and knowledge recommendation with machine learning,” *Sensors*, vol. 23, no. 1, p. 138, Dec. 2022, doi: 10.3390/s23010138.
- [6] L. Melendy, S. C. Hagen, F. B. Sullivan, T. R. H. Pearson, S. M. Walker, P. Ellis, A. K. Sambodo, O. Roswintiarti, M. A. Hanson, A. W. Klassen, M. W. Palace, B. H. Braswell, and G. M. Delgado, “Automated method for measuring the extent of selective logging damage with airborne LiDAR data,” *ISPRS J. Photogramm. Remote Sens.*, vol. 139, pp. 228–240, May 2018, doi: 10.1016/j.isprsjprs.2018.02.022.
- [7] L. A. Silva, H. S. S. Blas, D. P. García, A. S. Mendes, and G. V. González, “An architectural multi-agent system for a pavement monitoring system with pothole recognition in UAV images,” *Sensors*, vol. 20, no. 21, p. 6205, Oct. 2020, doi: 10.3390/s20216205.
- [8] M. Guerrieri and G. Parla, “Flexible and stone pavements distress detection and measurement by deep learning and low-cost detection devices,” *Eng. Failure Anal.*, vol. 141, Nov. 2022, Art. no. 106714, doi: 10.1016/j.engfailanal.2022.106714.
- [9] D. Jeong, “Road damage detection using YOLO with smartphone images,” in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 5559–5562, doi: 10.1109/BIGDATA50022.2020.9377847.

- [10] M. Izadi, A. Mohammadzadeh, and A. Haghighattalab, “A new neuro-fuzzy approach for post-earthquake road damage assessment using GA and SVM classification from QuickBird satellite images,” *J. Indian Soc. Remote Sens.*, vol. 45, no. 6, pp. 965–977, Mar. 2017.
- [11] Y. Bhatia, R. Rai, V. Gupta, N. Aggarwal, and A. Akula, “Convolutional neural networks based potholes detection using thermal imaging,” *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 34, no. 3, pp. 578–588, Mar. 2022, doi: 10.1016/j.jksuci.2019.02.004.
- [12] J. Guan, X. Yang, L. Ding, X. Cheng, V. C. Lee, and C. Jin, “Automated pixel-level pavement distress detection based on stereo vision and deep learning,” *Automat. Constr.*, vol. 129, p. 103788, Sep. 2021, doi: 10.1016/j.autcon.2021.103788.
- [13] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, and Y. Sekimoto, “RDD2022: A multi-national image dataset for automatic road damage detection,” 2022, arXiv:2209.08538.
- [14] J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 6517–6525, doi: 10.1109/CVPR.2017.690.
- [15] J. Redmon and A. Farhadi. YOLOv3: An Incremental Improvement. [Online]. Available: <https://pjreddie.com/yolo/>
- [16] A. Bochkovskiy, C.-Y. Wang, and H.-Y. Mark Liao, “YOLOv4: Optimal speed and accuracy of object detection,” 2020, arXiv:2004.10934.
- [17] G. Jocher, A. Chaurasia, A. Stoken, J. Borovec, Y. Kwon, K. Michael, J. Fang, C. Wong, D. Montes, Z. Wang, C. Fati, J. Nadar, V. Sonck, P. Skalski, A. Hogan, D. Nair, M. Strobel, and M. Jain, “Ultralytics/YOLOv5: V7.0—YOLOv5 SOTA realtime instance segmentation,” Zenodo, Tech. Rep., Nov. 2022. [Online]. Available: <https://zenodo.org/record/7347926>
- [18] C.-Y. Wang, A. Bochkovskiy, and H.-Y. Mark Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” 2022, arXiv:2207.02696.
- [19] R. Ali, D. Kang, G. Suh, and Y.-J. Cha, “Real-time multiple damage mapping using autonomous UAV and deep faster region-based neural networks for GPS-denied structures,” *Autom. Construct.*, vol. 130, Oct. 2021, Art. no. 103831. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S092658052100282X>
- [20] D. Kang and Y.-J. Cha, “Autonomous UAVs for structural health monitoring using deep learning and an ultrasonic beacon system with geo-tagging: Autonomous UAVs for SHM,” *Comput.-Aided Civil Infrastruct. Eng.*, vol. 33, no. 10, pp. 885–902, Oct. 2018.

- [21] Z. Xu, H. Shi, N. Li, C. Xiang, and H. Zhou, “Vehicle detection under UAV based on optimal dense YOLO method,” in *Proc. 5th Int. Conf. Syst. Informat. (ICSAI)*, Nov. 2018, pp. 407–411, doi: 10.1109/ICSAI.2018.8599403.
- [22] P. Kannadaguli, “YOLO v4 based human detection system using aerial thermal imaging for UAV based surveillance applications,” in *Proc. Int. Conf. Decis. Aid Sci. Appl. (DASA)*, Nov. 2020, pp. 1213–1219, doi: 10.1109/DASA51403.2020.9317198.
- [23] T. Petso, R. S. Jamisola, D. Mpoeleng, and W. Mmereki, “Individual animal and herd identification using custom YOLO v3 and v4 with images taken from a UAV camera at different altitudes,” in *Proc. IEEE 6th Int. Conf. Signal Image Process. (ICSIP)*, Oct. 2021, pp. 33–39, doi: 10.1109/ICSIP52628.2021.9688827.
- [24] L. Wang and Z. Zhang, “Automatic detection of wind turbine blade surface cracks based on UAV-taken images,” *IEEE Trans. Ind. Electron.*, vol. 64, no. 9, pp. 7293–7303, Sep. 2017, doi: 10.1109/TIE.2017.2682037.
- [25] D. Sadykova, D. Pernebayeva, M. Bagheri, and A. James, “IN-YOLO: Real-time detection of outdoor high voltage insulators using UAV imaging,” *IEEE Trans. Power Del.*, vol. 35, no. 3, pp. 1599–1601, Jun. 2020, doi: 10.1109/TPWRD.2019.2944741.
- [26] M. A. A. Khan, M. Alsawwaf, B. Arab, M. AlHashim, F. Almashharawi, O. Hakami, S. O. Olatunji, and M. Farooqui, “Road damages detection and classification using deep learning and UAVs,” in *Proc. 2nd Asian Conf. Innov. Technol. (ASIANCON)*, Aug. 2022, pp. 1–6, doi: 10.1109/ASIANCON55314.2022.9909043.
- [27] Y.-J. Cha, W. Choi, and O. Büyüköztürk, “Deep learning-based crack damage detection using convolutional neural networks,” *Comput.-Aided Civil Infrastruct. Eng.*, vol. 32, no. 5, pp. 361–378, May 2017.
- [28] M. Büyük, R. Duvar, and O. Urhan, “Deep learning based vehicle detection with images taken from unmanned air vehicle,” in *Proc. Innov. Intell. Syst. Appl. Conf. (ASYU)*, Oct. 2020, pp. 1–4, doi: 10.1109/ASYU50717.2020.9259868.
- [29] R. Li, J. Yu, F. Li, R. Yang, Y. Wang, and Z. Peng, “Automatic bridge crack detection using unmanned aerial vehicle and faster R-CNN,” *Construct. Building Mater.*, vol. 362, Jan. 2023, Art. no. 129659, doi: 10.1016/j.conbuildmat.2022.129659.
- [30] Y.-J. Cha, W. Choi, G. Suh, S. Mahmoudkhani, and O. Büyüköztürk, “Autonomous structural visual inspection using region-based deep learning for detecting multiple damage types,” *Comput.-Aided Civil Infrastruct. Eng.*, vol. 33, no. 9, pp. 731–747, Sep. 2018.

- [31] D. Kang, S. S. Benipal, D. L. Gopal, and Y.-J. Cha, “Hybrid pixel-level concrete crack segmentation and quantification across complex backgrounds using deep learning,” *Autom. Construct.*, vol. 118, Oct. 2020, Art. no. 103291.
- [32] S. Shim, J. Kim, S.-W. Lee, and G.-C. Cho, “Road damage detection using super-resolution and semi-supervised learning with generative adversarial network,” *Autom. Construct.*, vol. 135, Mar. 2022, Art. no. 104139, doi: 10.1016/j.autcon.2022.104139.
- [33] D. H. Kang and Y.-J. Cha, “Efficient attention-based deep encoder and decoder for automatic crack segmentation,” *Struct. Health Monitor.*, vol. 21, no. 5, pp. 2190–2205, Sep. 2022.
- [34] R. Ali and Y.-J. Cha, “Attention-based generative adversarial network with internal damage segmentation using thermography,” *Autom. Construct.*, vol. 141, Sep. 2022, Art. no. 104412.
- [35] W. Choi and Y. Cha, “SDDNet: Real-time crack segmentation,” *IEEE Trans. Ind. Electron.*, vol. 67, no. 9, pp. 8016–8025, Sep. 2020.
- [36] D. Arya, H. Maeda, S. Kumar Ghosh, D. Toshniwal, H. Omata, T. Kashiya, and Y. Sekimoto, “Global road damage detection: State-of-the-art solutions,” in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 5533–5539, doi: 10.1109/BIGDATA50022.2020.9377790.
- [37] V. Pham, C. Pham, and T. Dang, “Road damage detection and classification with Detectron2 and faster R-CNN,” in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2020, pp. 5592–5601, doi: 10.1109/BIGDATA50022.2020.9378027.
- [38] L. Parameswaran, “Deep learning based detection of potholes in Indian roads using YOLO,” in *Proc. Int. Conf. Inventive Comput. Technol. (ICICT)*, Feb. 2020, pp. 381–385, doi: 10.1109/ICICT48043.2020.9112424.
- [39] Y. Liu, G. Shi, Y. Li, and Z. Zhao, “M-YOLO based detection and recognition of highway surface oil filling with unmanned aerial vehicle,” in *Proc. 7th Int. Conf. Intell. Comput. Signal Process. (ICSP)*, Apr. 2022, pp. 1884–1887, doi: 10.1109/ICSP54964.2022.9778782.
- [40] Y. O. Ouma and M. Hahn, “Pothole detection on asphalt pavements from 2D-colour pothole images using fuzzy c-means clustering and morphological reconstruction,” *Autom. Construct.*, vol. 83, pp. 196–211, Nov. 2017, doi: 10.1016/j.autcon.2017.08.017.
- [41] M. Abdellatif, H. Peel, A. G. Cohn, and R. Fuentes, “Pavement crack detection from hyperspectral images using a novel asphalt crack index,” *Remote Sens.*, vol. 12, no. 18, pp. 1–10, 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/12/18/3084>

- [42] F. Viel, R. C. Maciel, L. O. Seman, C. A. Zeferino, E. A. Bezerra, and V. R. Q. Leithardt, “Hyperspectral image classification: An analysis employing CNN, LSTM, transformer, and attention mechanism,” *IEEE Access*, vol. 11, pp. 24835–24850, 2023.
- [43] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal, H. Omata, T. Kashiya, and Y. Sekimoto, “Crowdsensing-based road damage detection challenge (CRDDC-2022),” 2022, arXiv:2211.11362.

ACCEPTANCE LETTER



INTERNATIONAL CONFERENCE ON **AI-DRIVEN SMART SYSTEMS AND UBIQUITOUS COMPUTING** **ICAUC 2026**

19-21, JANUARY 2026

<https://guauc.com>
confguauc@gmail.com

Acceptance Letter

Paper ID	Paper Title	Author's Name
ICAUC-723	A Deep Learning Framework for Automated Road Surface Damage Detection Using UAV Imagery	Shashank Shatraboina, Vinod Upadhyay, Mulla Akbar Shehazad, Praveen Chouksey

Dear Author,

Greetings from ICAUC 2026!!

The Organizing Committee is pleased to inform you that the above peer-reviewed & refereed conference paper has been accepted for presentation at the **ICAUC 2026: International Conference on AI-Driven Smart Systems and Ubiquitous Computing** held at Shinawatra University, Bangkok, Thailand. on 19-21, January 2026.

In this regard, we appreciate if you could send the final paper, IEEE Copyright Form and Payment proof to the conference at the earliest, to ensure a timely publication of your research paper. When submitting your final paper, please highlight the changes made to the research paper according to the specified reviewer comments.

With Thanks,
Yours' Sincerely



Conference Chair
Prof. Dr. Pratikshya Bhandari
ICAUC 2026

PAYMENT RECEIPT



INTERNATIONAL CONFERENCE ON
AI-DRIVEN SMART SYSTEMS AND UBIQUITOUS COMPUTING
ICAUC 2026
THAILAND, 19-21 JANUARY 2026

<https://icauc.com>
conficauc@gmail.com

Payment Receipt

Receipt No. *ICAUC-2026-189*

Article Title *A Deep Learning Framework for Automated Road Surface Damage Detection Using UAV Imagery*

Paid By *Shashank Shatraboina, Vinod Upadhyay, Mulla Akbar Shehazad, Praveen Chouksey*

Amount ₹ 14,400/-
Rupees Fourteen Thousand Four Hundred only




Dr. Pratikshya Bhandari
Organising Chair

CERTIFICATE OF PRESENTATION







COPYRIGHT RECEIPT

IEEE COPYRIGHT AND CONSENT FORM

To ensure uniformity of treatment among all contributors, other forms may not be substituted for this form, nor may any wording of the form be changed. This form is intended for original material submitted to the IEEE and must accompany any such material in order to be published by the IEEE. Please read the form carefully and keep a copy for your files.

A Deep Learning Framework for Automated Road Surface Damage Detection Using UAV Imagery

Shashank Shatraboia, Vinod Upadhyay, Mulla Akbar Shehazad, Dr. Praveen Chouksey

2026 International Conference on AI-Driven Smart Systems and Ubiquitous Computing (ICAUC)

COPYRIGHT TRANSFER

The undersigned hereby assigns to The Institute of Electrical and Electronics Engineers, Incorporated (the "IEEE") all rights under copyright that may exist in and to: (a) the Work, including any revised or expanded derivative works submitted to the IEEE by the undersigned based on the Work; and (b) any associated written or multimedia components or other enhancements accompanying the Work.

GENERAL TERMS

1. The undersigned represents that he/she has the power and authority to make and execute this form.
2. The undersigned agrees to indemnify and hold harmless the IEEE from any damage or expense that may arise in the event of a breach of any of the warranties set forth above.
3. The undersigned agrees that publication with IEEE is subject to the policies and procedures of the [IEEE PSPB Operations Manual](#).
4. In the event the above work is not accepted and published by the IEEE or is withdrawn by the author(s) before acceptance by the IEEE, the foregoing copyright transfer shall be null and void. In this case, IEEE will retain a copy of the manuscript for internal administrative/record-keeping purposes.
5. For jointly authored Works, all joint authors should sign, or one of the authors should sign as authorized agent for the others.
6. The author hereby warrants that the Work and Presentation (collectively, the "Materials") are original and that he/she is the author of the Materials. To the extent the Materials incorporate text passages, figures, data or other material from the works of others, the author has obtained any necessary permissions. Where necessary, the author has obtained all third party permissions and consents to grant the license above and has provided copies of such permissions and consents to IEEE

You have indicated that you DO wish to have video/audio recordings made of your conference presentation under terms and conditions set forth in "Consent and Release."

CONSENT AND RELEASE

1. In the event the author makes a presentation based upon the Work at a conference hosted or sponsored in whole or in part by the IEEE, the author, in consideration for his/her participation in the conference, hereby grants the IEEE the unlimited, worldwide, irrevocable

permission to use, distribute, publish, license, exhibit, record, digitize, broadcast, reproduce and archive, in any format or medium, whether now known or hereafter developed: (a) his/her presentation and comments at the conference; (b) any written materials or multimedia files used in connection with his/her presentation; and (c) any recorded interviews of him/her (collectively, the "Presentation"). The permission granted includes the transcription and reproduction of the Presentation for inclusion in products sold or distributed by IEEE and live or recorded broadcast of the Presentation during or after the conference.

2. In connection with the permission granted in Section 1, the author hereby grants IEEE the unlimited, worldwide, irrevocable right to use his/her name, picture, likeness, voice and biographical information as part of the advertisement, distribution and sale of products incorporating the Work or Presentation, and releases IEEE from any claim based on right of privacy or publicity.

BY TYPING IN YOUR FULL NAME BELOW AND CLICKING THE SUBMIT BUTTON, YOU CERTIFY THAT SUCH ACTION CONSTITUTES YOUR ELECTRONIC SIGNATURE TO THIS FORM IN ACCORDANCE WITH UNITED STATES LAW, WHICH AUTHORIZES ELECTRONIC SIGNATURE BY AUTHENTICATED REQUEST FROM A USER OVER THE INTERNET AS A VALID SUBSTITUTE FOR A WRITTEN SIGNATURE.

Shashank Shatraboina

08-01-2026

Signature

Date (dd-mm-yyyy)

Information for Authors

AUTHOR RESPONSIBILITIES

The IEEE distributes its technical publications throughout the world and wants to ensure that the material submitted to its publications is properly available to the readership of those publications. Authors must ensure that their Work meets the requirements as stated in section 8.2.1 of the IEEE PSPB Operations Manual, including provisions covering originality, authorship, author responsibilities and author misconduct. More information on IEEE's publishing policies may be found at http://www.ieee.org/publications_standards/publications/rights/authorrightsresponsibilities.html Authors are advised especially of IEEE PSPB Operations Manual section 8.2.1.B12: "It is the responsibility of the authors, not the IEEE, to determine whether disclosure of their material requires the prior consent of other parties and, if so, to obtain it." Authors are also advised of IEEE PSPB Operations Manual section 8.1.1B: "Statements and opinions given in work published by the IEEE are the expression of the authors."

RETAINED RIGHTS/TERMS AND CONDITIONS

- Authors/employers retain all proprietary rights in any process, procedure, or article of manufacture described in the Work.
- Authors/employers may reproduce or authorize others to reproduce the Work, material extracted verbatim from the Work, or derivative works for the author's personal use or for company use, provided that the source and the IEEE copyright notice are indicated, the copies are not used in any way that implies IEEE endorsement of a product or service of any employer, and the copies themselves are not offered for sale.
- Although authors are permitted to re-use all or portions of the Work in other works, this does not include granting third-party requests for reprinting, republishing, or other types of re-use. The IEEE Intellectual Property Rights office must handle all such third-party requests.
- Authors whose work was performed under a grant from a government funding agency are free to fulfill any deposit mandates from

that funding agency.

AUTHOR ONLINE USE

- **Personal Servers.** Authors and/or their employers shall have the right to post the accepted version of IEEE-copyrighted articles on their own personal servers or the servers of their institutions or employers without permission from IEEE, provided that the posted version includes a prominently displayed IEEE copyright notice and, when published, a full citation to the original IEEE publication, including a link to the article abstract in IEEE Xplore. Authors shall not post the final, published versions of their papers.
- **Classroom or Internal Training Use.** An author is expressly permitted to post any portion of the accepted version of his/her own IEEE-copyrighted articles on the author's personal web site or the servers of the author's institution or company in connection with the author's teaching, training, or work responsibilities, provided that the appropriate copyright, credit, and reuse notices appear prominently with the posted material. Examples of permitted uses are lecture materials, course packs, e-reserves, conference presentations, or in-house training courses.
- **Electronic Preprints.** Before submitting an article to an IEEE publication, authors frequently post their manuscripts to their own web site, their employer's site, or to another server that invites constructive comment from colleagues. Upon submission of an article to IEEE, an author is required to transfer copyright in the article to IEEE, and the author must update any previously posted version of the article with a prominently displayed IEEE copyright notice. Upon publication of an article by the IEEE, the author must replace any previously posted electronic versions of the article with either (1) the full citation to the IEEE work with a Digital Object Identifier (DOI) or link to the article abstract in IEEE Xplore, or (2) the accepted version only (not the IEEE-published version), including the IEEE copyright notice and full citation, with a link to the final, published article in IEEE Xplore.

Questions about the submission of the form or manuscript must be sent to the publication's editor.

Please direct all questions about IEEE copyright policy to:

IEEE Intellectual Property Rights Office, copyrights@ieee.org, +1-732-562-3966